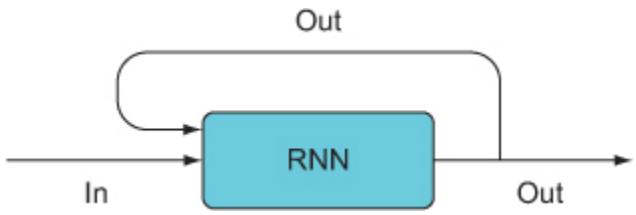
Figure 5.7. A generic recurrent neural network (RNN) layer processes a sequence of data by incorporating its previous output with the new input. The input on the left, along with a previous output is fed into an RNN module, which then produces an output. The output is fed back into the RNN on the next time step, and a copy may be fed into another layer. An RNN will not work properly with single experiences in an experience replay buffer since it needs to work on sequences of experiences.



One way to use RNNs without an experience replay is to run multiple copies of the agent in parallel, each with separate instantiations of the environment. By distributing multiple independent agents across different CPU processes (figure 5.8), we can collect a varied set of experiences and therefore get a sample of gradients that we can average together to get a lower variance mean gradient. This eliminates the need for experience replay and allows us to train an algorithm in a completely online fashion, visiting each state only once as it appears in the environment.

Figure 5.8. The most common form of training a deep learning model is to feed a batch of data together into the model to return a batch of predictions. Then we compute the loss for each prediction and average or sum all the losses before backpropagating and updating the model parameters. This averages out the variability present across all the experiences. Alternatively, we can run multiple models with each taking a single experience and making a single prediction, backpropagate through each model to get the gradients, and then sum or average the gradients before making any parameter updates.